

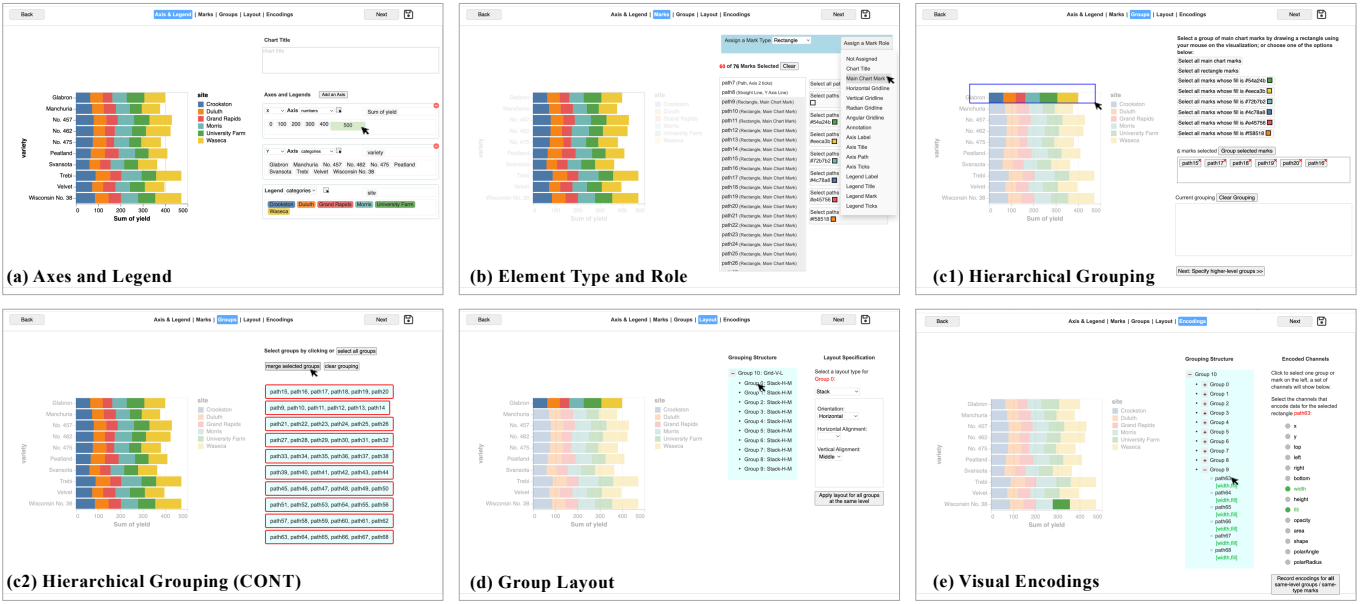


Fig. 5: The labeling tool we have developed to produce the semantics labels in VISANATOMY divides the labeling process into five stages: **Axis&Legend** (a), **Marks** (b), **Groups** (c1, c2), **Layout** (d), and **Encodings** (e) and provides necessary interactions to accelerate the labeling process.

elements in the label file. Axes and legends contain information about the type (e.g., x, y, angle, radius) and orientation (e.g., horizontal, vertical), and they are further broken down into lower-level components such as labels and ticks that are also specified with IDs of their corresponding elements. Figure 1(c-II) shows the semantic labels for the x-axis, y-axis, and the color legend in the chart.

Element Type and Role record the shape type and semantic role of each SVG element. In SVG files, the tag name of an element does not always match the geometric shape. VISANATOMY resolves this ambiguity through the **Element Type** label. For example, in Figure 1(c-III), “path64”, the bar representing “Winsconsin No. 38” in Waseca, has the tag name “path”, while its **Element Type** is labeled as **Rectangle**. In addition, the same type of elements can play different roles in a chart. For example, elements “path64” and “path82” are both rectangles in Figure 1(c-III), but the former is labeled with the **Element Role** **Main Chart Mark** and the latter is labeled as a **Legend Mark**.

Hierarchical Grouping reflects the multi-level semantic clustering of main chart marks (elements whose **Element Role** is **Main Chart Mark**, i.e., the 60 colored bars from the main chart area). The example stacked bar chart has 10 mark groups (“Group1” to “Group10”), each corresponding to one of the 10 varieties of barley (i.e., “Glabron” to “Winsconsin No. 38” along the y-axis), and they further form a higher-level group, “Group11”, that encapsulates all the 10 lower-level groups. This hierarchy is recorded in the label file as shown in Figure 1(c-IV).

Group Layout indicates the spatial relationship between visual objects within one group (e.g., grid, stack, packing, radial) with orientation (e.g., horizontal, vertical, angular) and alignment parameters (e.g., bottom-aligned, left-aligned). This information is present for all the groups at all levels. For example, in Figure 1(c-V), “Group1” is labeled with a middle-aligned horizontal stack layout, and the same layout applies across “Group2” to “Group10”; the higher-level “Group11” is labeled as a left-aligned vertical grid layout.

Visual Encodings records which visual elements and channels are used to encode data. Figure 1(c-VI) shows labels for visual encodings in the stacked bar chart, indicating that the bar “width” and “fill” encode data values. In some charts, the visual channels of a group can encode data as well. For example, in a small-multiple design of grouped bar charts, the position of each bar chart can encode the approximate geographic location of the corresponding U.S. state. VISANATOMY thus organizes visual encoding labels by each visual object’s ID.

These semantic components are used as fundamental building

blocks in programming libraries such as Mascot.js [48] and authoring tools like Chartulator [62]. They are expressive enough to describe not only standard charts but also bespoke designs, as evidenced by these tools’ galleries. Our labeling effort, described in the next section, confirms the expressivity of these components.

3.2 Labeling with A Semi-Automated Tool

These semantic labels are created using a semi-automated chart labeling tool we have developed. According to Chen and Liu [16], obtaining high-quality chart labels is expensive and time-consuming, especially for complex labels that require careful examination of charts. Moreover, SVG charts have diverse hierarchical structures and utilize SVG elements in various ways, especially when they come from different tools and sources. To address these challenges, we have developed a mixed-initiative multi-stage labeling tool to facilitate the process and mitigate laborious inspection.

The annotation tool divides the labeling process into five stages: **Axis&Legend**, **Marks**, **Groups**, **Layout**, and **Encodings**. Figure 5 shows the system user interface and an example labeling workflow using the stacked bar chart in Figure 1(c). Upon loading the chart, the system traverses the SVG hierarchy to obtain **All Graphic Primitives**. Then the system leverages the heuristics-based methods reported in the Mystique system [15] to detect the chart title, axes, and legend, and displays the results in the **Axis&Legend** UI accordingly (Figure 5(a)). The annotator can correct the results through drag-and-drop or lasso selection over SVG texts to revise **Reference Elements**; for example, in Figure 5(a), the annotator is dragging the text “500” from the chart into the x-axis label box. The annotator can also add more axes if more than two axes exist (e.g., in parallel coordinates).

Once the annotator finishes inspecting the results in this stage, they can click the “Next” button to go to the **Marks** stage, where the full list of graphical elements is displayed (Figure 5(b)). The annotator can click to select a single mark, batch-select multiple marks through the generalized selection options [32] (e.g., selecting the same-type or same-color marks), or shift-click to select consecutive marks in the mark list. When a set of marks is selected, they will be highlighted in full opacity in the chart, while all other elements will be partially transparent. The annotator can label the **Element Type and Role** of the selected marks through the corresponding drop-down menus. In Figure 5(b), the selected paths are assigned **Element Type** **Rectangle** and **Element Role** **Main Chart Mark**. At this stage, the annotator would also need to

specify gridlines and low-level components for axes and legend (e.g., ticks, paths), so that the [Reference Elements](#) label is complete.

In the next stage, **Groups**, the annotator specifies [Hierarchical Grouping](#) on all the main chart marks. Since the use of the `<g>` tag is inconsistent across charts produced by different tools [15, 16], the original SVG grouping information is ignored by the labeling tool. To select marks for grouping, the annotator can click to select individual elements, make a lasso selection (Figure 5(c1)), or choose from the generalized selection options. The selected marks can then be grouped by clicking the “Group selected marks” button. After that, the system will recommend a list of inferred “other mark groups”, and the annotator can either agree to finish the lowest-level grouping or disagree to continue grouping manually. Once all lowest-level mark groups are specified, the annotator clicks the “Specify higher-level groups” button to go to the second phase of the **Groups** UI, where they work on higher-level grouping, e.g., in Figure 5(c2) the annotator has selected all 10 mark groups and are merging them into one final group.

After [Hierarchical Grouping](#) is finished, the annotator goes to the **Layout** stage. Here the hierarchical groups will be displayed as a collapsible nested list with clickable items. The annotator can click on an individual group and label its layout type and parameters, as shown in Figure 5(d). In the final **Encodings** stage, the system adds mark items to the nested list and lets the annotator specify their encoded visual channels (Figure 5(e)). The visual channel list will be updated accordingly based on [Element Type](#) of the selected element. In both the **Layout** and **Encodings** stages, the annotator can apply the label of one group or mark to its peers, i.e., same-level and same-type objects, to accelerate the process of assigning [Group Layout](#) and [Visual Encodings](#).

At any time during the labeling process, the annotator can click on the “save” button in the upper-right corner of the interface to save a local copy of all semantic labels created so far in the JSON format. When the same chart is loaded next time, the system will automatically retrieve the corresponding label file (if it exists), and load the semantic labels in the UI accordingly to allow the annotator to inspect existing labels and continue unfinished work. The semi-automated labeling system and a document recording the options for [Element Type](#), [Element Role](#), types and parameters of [Group Layout](#), and channels of [Visual Encodings](#), are included in the supplementary materials².

²see the [code and details](#) for the labeling system.

3.3 Iterative Annotation and Quality Control

Using the system to produce high-quality semantic labels requires sufficient knowledge and expertise in the visualization field. Thus, we adopt an iterative approach to obtain the labels from three experts who are familiar with visualization abstraction papers in the team. The first author, a Ph.D. student who has published peer-reviewed papers in visualization-related conferences (e.g., VIS, EuroVis), performed the first-round labeling. The time required to finish labeling one chart ranges between 10 to 20 minutes. After that, the second author, a Ph.D. student who has published at visualization-related conferences such as VIS and IUI, and the last author, a researcher who has been contributing to the visualization community for more than 15 years, performed the second-round inspection. Each chart has been reviewed by at least two experts on the team. Whenever disagreements over certain semantic labels arose, the three authors discussed the cases to reach a consensus on the correct labeling approach, and all the charts with features related to the issues would be re-labeled. These discussions not only improved the consistency and accuracy of the labels but also enhanced the labeling system.

3.4 Comparing VISANATOMY with Related Corpora

In this section, we compare VISANATOMY with nine existing chart corpora: Beagle [8], YOLOt++ [29], REV [61], MASSIVE [9], MVV [19], VisImages [27], Chart-LLM [44], VisText [71], and VisEval [18] (results are shown in Table 1). These corpora were selected based on the following criteria: the corpus should be publicly available, contain chart images, and have been published at VIS or HCI venues. Based on the criteria, we did not include the Visually29k corpus [50] as it focuses on infographics and the VIS30K corpus [17] which contains images of data tables.

VISANATOMY distinguishes from other chart corpora in two major aspects: the rich, multi-granular chart semantic labels, and its diversity regarding chart types, designs, and sources. In terms of the number of charts, VISANATOMY is comparable to datasets that specifically emphasize label quality, such as VisEval [18] and Chart-LLM [44], which contain around one or two thousand samples. Although corpora such as Beagle [8] collect many more charts through web crawling, they tend to include duplicate designs with unbalanced chart distributions [15]. Moreover, they lack fine-grained component-level labels. On the other hand, corpora created through computer-aided generation (e.g., YOLOt++ [29]) do provide fine-grained component labels, but are highly restricted in terms of chart and tool diversity.

Table 1: A comparison between VISANATOMY and nine related chart corpora in their current states. ✓ indicates the existence of a certain property in the corresponding corpus, ✓ indicates partial existence, ✗ means a property is absent in the current state but can be obtained with some effort, and - indicates that a property is unavailable.

	VisAnatomy	MVV [19]	VisEval [18]	YOLOt++ [29]	REV [61]	VisText[71]	Beagle [8]	MASSIVE [9]	VisImages [27]	Chart-LLM [44]
Primary Collection Method	Manual Curation	Manual Curation	Transforming An Existing Corpus	Computer-Aided Generation	Computer-Aided Generation	Computer-Aided Generation	Web Crawling	Web Crawling	Web Crawling	Web Crawling
# Charts	946	360	1,150	15,197	5,125	12,441	~41,000	2,070	12,267	1,981
# Types	40	14	7	11	4	3	24	12	34	10
# Tools	58	-	1	2	-	1	5	-	-	1
Format	SVG	✓	-	✓	✓	✗	✓	-	-	✗
	Bitmap	✓	✓	✗	✓	✓	✓	✓	✓	✗
	Program	-	-	✓	-	✓	-	-	-	✓
Label	Chart Type	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Chart BBox	✓	✓	-	-	-	-	-	✓	-
	Element BBox	✓	-	-	✓	✓	-	-	-	-
Label	Legend Elements	✓	-	-	✓	✗	-	-	-	-
	Axis Elements	✓	-	-	✓	✓	-	-	-	-
	Element Shape	✓	-	-	✓	✗	-	-	-	-
Label	Element Role	✓	-	-	✓	✓	-	-	-	-
	Mark Grouping	✓	-	-	-	-	-	-	-	-
	Group Layout	✓	-	-	-	-	-	-	-	-
Label	Visual Encoding	✓	-	-	-	-	-	-	-	✗
	(NL, VIS) pairs	-	-	✓	-	✓	-	-	-	✓

VISANATOMY balances between promoting diversity for real-world charts and maintaining quality control for fine-grained labels. The labels for over 380K visual elements in VISANATOMY adequately support applications requiring extensive component-level annotations.

4 USE CASES

To showcase the utility of VISANATOMY, we present four use cases, each focusing on a specific downstream visualization application:

- *Inferring semantic roles of SVG elements* with Large Language Models (LLMs): we evaluate two LLMs to examine their capabilities of classifying SVG elements into component types.
- *Decomposing rectangle-based SVG charts for layout reuse*: we use VISANATOMY to obtain a validation set to evaluate the performance of an existing system Mystique [15].
- *Classifying chart types* using Graph Neural Networks (GNNs) [76] and vision models: we report and compare the performance of vision models and graph models using VISANATOMY as a benchmark corpus, and discuss the trade-offs involved.
- *Supporting accessible navigation* of chart content for visually impaired people: we demonstrate how VISANATOMY can enable the replication of the rich screen reader experiences through keyboard navigation as described in the study by Zong et al. [79].

The four use cases span different requirements on the semantic labels, from low-level mark details (e.g., SVG element role) to high-level chart information (e.g., chart type). Through these applications, we demonstrate how VISANATOMY can enable tighter integration of AI in visualization. In addition, we illustrate how the semantic labels in VISANATOMY ease the constraints on input formats such as the necessity for charts to be created with specific tools, supporting a wider range of input charts. The supplementary materials³ include our implementations, detailed results, and demo videos for the use cases.

4.1 Semantic Role Inference with LLMs

Inferring chart semantics is a classic task in automated visualization understanding. For instance, ReVision [66] utilizes vision models to detect the mark types and the underlying data used in a bitmap visualization. More recently, with the rapid advancement of LLMs, researchers have found that (1) text-based chart specifications improve the performance of language models (LMs) on chart-reading tasks compared to vision-based approaches [12], and (2) the existence of semantic information for SVG elements (so called “Primal Visual Description (PVD)”) boost the performance of LMs for vector graphics reasoning [74]. In this use case, we would like to examine the capabilities of current LLMs in semantic inference; more specifically, we focus on the task of classifying SVG elements into the following categories: main chart elements (marks), axes (titles, labels), and legend (titles, labels, marks). This classification is an important prerequisite for many downstream tasks [15, 61], and we evaluate it using one open-source LLM (DeepSeek-V3-0324 [47]) and one proprietary LLM (GPT-4o [38]).

Prompt. The prompt contains the role specification (“*expert in analyzing SVG-based data visualizations*”), the task (“*identify and categorize different visual elements in the provided SVG chart into main chart marks, legend components (title, mark, label), and axis components (title, label)*”), explanations on the semantic categories, and the required JSON-format output. For both models, we utilized their cloud APIs for inference due to their significant large model sizes.

Results. DeepSeek-V3 successfully processed 772 charts and GPT-4o processed 835, according to their respective context length constraints (i.e., input token limits of 64K for DeepSeek-V3 and 128K for GPT-4o). To make it a fair comparison, we report results from the 772 charts processed by both models. We compare their inferences with the ground truth annotations from VISANATOMY (Main Chart Marks and Reference Elements), and record F1, precision, and recall scores for each semantic category.

³see our [implementations, results, and demo videos](#) for the use cases.

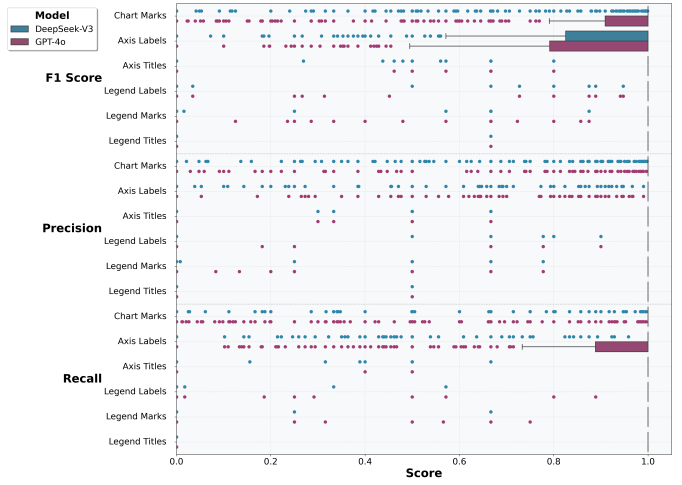


Fig. 6: Performance of two LLMs in inferring the semantic roles of SVG elements for 772 charts. The whiskers show $1.5 \times \text{IQR}$ thresholds, and the dots are considered “outlier” charts.

The two LLMs achieve comparable performances in terms of the overall F1 scores: DeepSeek-V3 (mean 0.841, median 0.870), GPT-4o (mean 0.839, median 0.929). Both models performed well in recognizing *axis titles*, *legend marks*, *legend labels*, and *legend titles*, with the maximum, median, and $1.5 \times \text{IQR}$ all equal to 1 (Figure 6). The performance of both models in inferring *main chart marks* and *axis labels* is comparatively lower than for the other categories, with DeepSeek-V3 demonstrating superior results to GPT-4o for these two categories. Based on this result, *the current state-of-the-art LLMs show promising performance in recognizing the semantic roles of SVG elements*, indicating a great potential to develop LLM-assisted systems for SVGs. In addition, *when working with these LLM inferences, careful human verifications and corrections need to be supported, especially for main chart marks and axis labels*.

We also examined model performance across different chart types, as these vary in design elements and complexity. The key finding here is that *GPT-4o tends to be more accurate on more complex charts compared to DeepSeek-V3, and vice versa*. For instance, DeepSeek-V3 achieves better mean F1 scores in traditional statistical visualizations such as simple bar charts (0.887 vs 0.543, 63% performance gap), area charts (0.876 vs 0.641, 37% gap), and stacked bar charts (0.902 vs 0.607, 49% gap), while GPT-4o exhibits superiority with lower margin in complex visualizations including circle packing diagrams (0.959 vs 0.885, 8% gap), bullet charts (0.850 vs 0.697, 22% gap), and bespoke visualizations (0.915 vs 0.846, 8% gap). These performance gaps may be attributed to potential differences between visualization training data used for these LLMs and suggest the need for fine-tuning for better results across chart types.

Finally, we analyzed model performance across charting tools, focusing on 18 tools that each contribute at least 10 charts, resulting in a total of 401 charts. (Note that tool information is unavailable for certain charts in VISANATOMY.) DeepSeek-V3 achieves a mean F1 above 0.9 for 16 of these tools, while GPT-4o does so for only 7 tools. AnyChart [4], FlexChart [72], AngularChart [6], Matplotlib [54], Semiotic [68], and Mascot.js [48] appear in the top performers for both models. The performance on charts created with the following tools shows a substantial discrepancy between the two models: R (0.692 for GPT-4o, 0.959 for DeepSeek-V3) and NIVO [59] (0.765 for GPT-4o, 0.960 for DeepSeek-V3). D3.js [11] presents a challenging case for both models (0.769 for GPT-4o, 0.863 for DeepSeek-V3), probably due to D3’s high versatility and expressivity.

4.2 Chart Layout Deconstruction

Unlike previous corpora that lack semantic information on low-level components, VISANATOMY specifies the mark grouping, layout, and data-channel encoding information, which play a vital role in the task